

CRISP: Contact-Rich Robotic Simulation Platform with Extensive Geometries and Contact Solvers

Somang Lee, Sunkyung Park, Jinhee Yun, Seoki An, and Dongjun Lee
Department of Mechanical Engineering, Seoul National University
Email: {hopelee,sunk1136,yjhs0932,seoki97s,djlee}@snu.ac.kr

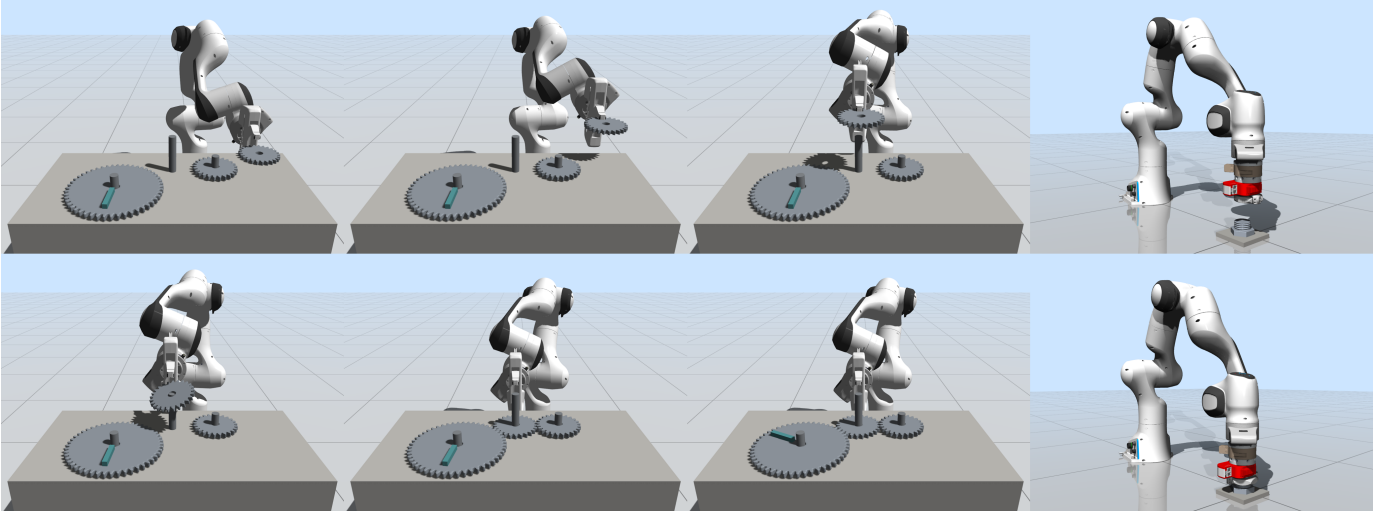


Fig. 1: Snapshots of demonstration scenarios using our simulator, CRISP. Left: Gear insertion and driving. Right: Bolt-nut assembly. Collision geometries are represented using meshes and SDFs for reliable collision detection, and intensive multi-contact interactions are simulated faster than real time using a robust and accurate contact solver.

Abstract—We present CRISP (Contact-Rich Simulation Platform), a high-fidelity physics engine tailored for complex multi-contact simulations such as tight-tolerance robotic manipulation. Achieving high physical fidelity in robotic simulation requires both expressive modeling of geometry and contact interactions, as well as accurate numerical resolution via robust collision detection and contact solvers. However, existing simulators often either rely on limited support for geometric representations and simplified modeling of contact interactions, or employ numerical resolution methods whose accuracy or robustness is inherently constrained. Accordingly, we develop a new simulator that supports diverse geometric representations with accurate optimization-based collision detection, and combines contact modeling with robust augmented Lagrangian-based contact solvers. This integration enables efficient and consistent detection of contact information across complex geometries while accurately resolving multi-contact constraints without problematic relaxations, which is essential for simulating contact-intensive and sharp interactions. We validate the physical fidelity of our simulator against state-of-the-art platforms and further demonstrate its capabilities through complex robotic manipulation scenarios. CRISP is publicly available at <https://github.com/INRoL/crisp>¹.

I. INTRODUCTION

Simulation has emerged as an indispensable component for the advancement of modern robotics, providing a safe, scalable, and cost-effective alternative to real-world experimentation. Following the development of diverse physics engines [9, 19, 23, 30, 40, 43, 45], they have been widely adopted as versatile platforms that facilitate algorithm verification, system identification, mechanical design, and synthetic data acquisition [7]. More recently, this utility has further evolved into serving as a foundational data generation platform for the advanced robotic intelligence [35, 36, 49]. In this paradigm, high physical fidelity is crucial to ensure that the learned policy internalizes the underlying dynamics required for reliable zero-shot transfer to the physical world.

Since simulation is fundamentally a process of modeling physical phenomena and numerically solving the resulting models, high physical fidelity critically depends on both the expressiveness of the physical model and the accuracy of its numerical resolution. In robotic simulation, particularly for contact-rich applications, these two aspects are largely governed by the underlying geometries and the handling of the contact interactions.

¹Project website: <https://inrol.github.io/crisp/>.

From the modeling perspective, physical fidelity is determined by the expressive scope of the simulator in representing geometry and contact interactions. In terms of geometric representations, many widely used engines do not natively support signed distance field (SDF) representations (see Table I), which provide a continuous description of geometry well suited for accurate contact modeling [29]. By contrast, Isaac Sim [30] supports SDF-based collision handling by processing imported meshes into voxelized distance fields, enabling scalable collision detection but limiting the use of custom analytic SDFs [8], whereas MuJoCo [45] allows user-defined SDF geometries with greater flexibility while not supporting collision handling between general non-convex meshes. Beyond geometric representations, existing simulators also substantially differ in modeling contact interactions. Formally, the constrained dynamics with contact is naturally modeled as a nonlinear complementarity problem (NCP) [42] which imposes the non-penetration and friction conditions; yet this formulation remains highly challenging to solve directly due to its non-smoothness and non-convexity [20]. Consequently, most contemporary simulators relax the original NCP into more tractable formulations to ensure stable performance. For instance, ODE [40], Bullet [9], and DART [23] linearize the friction cone to formulate a linear complementarity problem (LCP). While this formulation is more tractable, such approximation violates isotropy, leading to undesirable biased friction [25]. Another popular approximation is to convexify the original NCP to a cone complementarity problem (CCP) [3, 44] by relaxing the Signorini condition (i.e., normal complementarity), which is adopted in MuJoCo and Drake [43]. While this convexification allows the use of powerful convex optimization techniques, it induces *gliding* artifacts during sliding [6]. On the other hand, Isaac Sim employs a position-based dynamics formulation that prioritizes numerical stability over strict physical consistency [33].

From the perspective of numerical resolution, the achievable physical fidelity of robotic simulation is further influenced by collision detection and contact constraint resolution. In particular, the geometric validity and consistency of collision detection are inherently constrained by the supported combinations of geometric representations. For instance, MuJoCo employs sampling-based and gradient-descent methods for SDF-SDF collision detection [38], which are effective for smooth convex geometries but can suffer from local minima in complex configurations. On the other hand, voxelized SDF in Isaac Sim enables robust collision detection between arbitrary shapes [29], yet the discretization may distort the original geometry and fail to capture fine-grained interactions [34]. Similarly, the choice of contact solvers is often constrained by the underlying contact modeling formulation. Many simulators, including ODE, Bullet, DART, and Isaac Sim, rely on projected Gauss-Seidel (PGS)-type first-order methods, which lack convergence guarantees for ill-conditioned or tightly coupled multi-contact constraints [48]. In contrast, MuJoCo and Drake formulate contact resolution as convex optimization problems and apply second-order methods such as Newton

Physics engine	Geometry ¹	Formulation	Solver
ODE [40]	Mesh	LCP	Direct/PGS
Bullet [9]	Mesh	LCP	Direct/PGS
DART [23]	Mesh	LCP	Direct/PGS
MuJoCo [45]	Mesh/SDF	CCP	Newton
Drake [43]	Mesh	CCP	Newton
RaiSim [19]	Mesh	²	Bisection
Isaac Sim [30]	Mesh/SDF ⁴	PBD	PGS/TGS
CRISP (ours)	Mesh/SDF/DSF	NCP	AL-variant

¹ Primitives and convexes are omitted; they are supported by all.

² RaiSim uses a CCP variant to enforce the Signorini condition.

³ MuJoCo convexifies meshes except for mesh-SDF pairs.

⁴ Isaac Sim utilizes internally processed voxelized SDFs.

TABLE I: Comparison of existing physics engines and CRISP.

method [6], enabling higher accuracy and improved convergence properties.

Motivated by these considerations, we present CRISP, a high-fidelity physics engine tailored for contact-rich robotic simulations. CRISP addresses geometric and contact modeling by supporting a diverse set of geometric representations, including meshes and function-based formulations, and by directly considering the original nonlinear complementarity formulation for both rigid and compliant contacts. These modeling capabilities are complemented by accurate optimization-based collision detection algorithms tailored to function-based geometric representations and a robust contact solver based on augmented Lagrangian (AL) theory [22]. This integration enables the efficient and consistent detection of numerous contact points across complex geometries, while accurately resolving tightly coupled multi-contact constraints without problematic relaxations, which is essential for simulating contact-intensive and sharp interactions. Our contributions are summarized as follows:

- To our knowledge, this is the first physics engine both capable of accurately capturing complex collision geometries and resolving multi-contact interactions without unphysical relaxations, by providing various optimization based collision detection algorithms and contact solvers that are highly accurate and efficient compared to the other approaches in existing simulators.
- Our framework is validated by complex manipulation demonstrations and comparative evaluation against state-of-the-art platforms, exhibiting superior physical fidelity.
- CRISP is publicly available, with documentation, examples, and videos provided on the project website.

The rest of the paper is organized as follows. In Sec. II, we review typical contact handling methods in existing physics engines. Sec. III provides a high-level overview of CRISP, with Sec. IV introducing its diverse geometric representations and Sec. V describing its AL-based contact solvers. Various comparative evaluations and demonstrations are presented in Sec. VI, followed by concluding discussions in Sec. VII.

II. RELATED WORKS

Contact interactions are typically managed through the interplay between collision detection and contact solving.

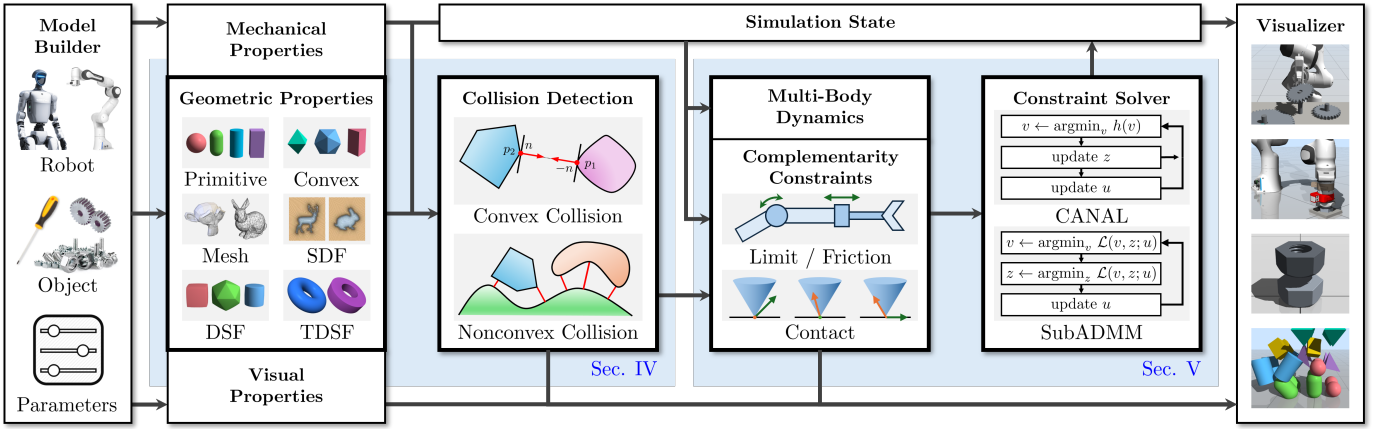


Fig. 2: **High-level architecture of CRISP.** The model builder defines the low-level structure `model_t`, from which `data_t` is instantiated. At each step, collision detection and constraint solving are performed, and the resulting state is visualized.

Table I summarizes the methodological choices of widely-used physics engines in robotics, alongside the proposed CRISP.

A. Collision Detection

Collision detection serves to identify the contact informations between geometries and construct the contact constraints. Among various geometric representations, primitive shapes (e.g., spheres, capsules, boxes) and convex polytopes are the most simulation-friendly form, as they enable explicit solutions or efficient algorithms for collision queries [14, 32, 41]. In contrast, while triangle meshes are the *de facto* standard for graphical purpose, their discrete and non-convex nature poses significant challenges for robust collision detection [46]. Consequently, most physics engines either offer restricted support (e.g., Drake) or rely on computationally expensive detection algorithms [37] for mesh collisions; thus strongly recommending convex decomposition [31, 47] as a pre-processing step.

From a simulation perspective, function-based representations offer several advantages over discrete meshes. Most notably, SDFs enable efficient and robust collision detection even in complex configurations [29]. However, the applicability of SDFs is often limited by the availability of analytic distance functions, and the efficient storage and querying of such representations remain challenging [13]. Another promising form is the differentiable support function (DSF) to represent smooth convex shapes, which is independently proposed in [16] and [21]. This representation provides second-order surface information with respect to the supporting direction, enabling fast and robust collision detection between smooth convex shapes [1].

B. Contact Solver

Since multi-contact NCP originating from rigid body dynamics with frictional contacts is inherently difficult to solve directly due to its non-smooth and non-convex nature [20], most contact solvers address relaxed formulations. The LCP formulation, which linearizes the friction cone into a pyramid,

allows the use of direct methods such as Lemke’s [26] and Dantzig’s algorithms, or per-contact iterative methods like PGS algorithm [17], which is widely adopted in game and graphics communities [2] as well as in several robotic simulators including ODE, Bullet, and DART, despite its poor convergence properties and the introduction of anisotropic friction [25]. Another widely used relaxation is the CCP formulation, whose convexification enables the used of powerful convex optimization techniques, such as Newton method, as adopted in MuJoCo and Drake. To mitigate the *gliding* artifacts associated with CCP, Raisim [19] adopts an alternative formulation, yet it introduces incorrect friction direction [25].

To avoid these relaxation-induced artifacts, there have been several attempts to address the original NCP. For instance, Howell et al. [18] adopted an interior point method and Macklin et al. [27] utilized a complementarity function, though both approaches eventually induce numerical smoothing. Macklin et al. [28] also proposed a variant of the PGS method later employed in Isaac Sim, which relies on the position-level dynamics formulation. More recently, AL methods have been explored to gradually address the multi-contact NCP by solving a sequence of relaxed subproblems. Carpentier et al. [5] introduced an ADMM-based solver using a dual formulation, while Lee et al. [22] proposed two variations of the AL method using a primal-dual formulation.

III. CRISP: CONTACT-RICH SIMULATION PLATFORM

A. System Overview

We develop CRISP to enable efficient and accurate simulation of contact-rich robotic systems across a wide range of manipulation scenarios. The overall architecture of CRISP is inspired by MuJoCo and structured into four primary components, as depicted in Fig. 2:

- The initial stage involves **model building**, where the user defines the simulation scenario by specifying the mechanical properties of bodies and joints, geometric properties attached to each body, and various solver parameters to tune the simulation fidelity.

- The **collision detection** engine identifies pairs of geometries where a collision is imminent or has already occurred under the current simulation state, extracting contact information such as penetration depths, contact points and normals.
- The **constraint solver** is a core component of the CRISP physics engine and is responsible for solving multi-body dynamics subject to contact and joint constraints; in particular, CRISP provides two tailored AL-based solvers, CANAL and SubADMM, ensuring both computational efficiency and physical accuracy.
- The **visualization** is handled by a built-in OpenGL-based forward renderer, integrated directly into the simulator to provide lightweight yet efficient visual feedback.

Our framework is implemented in C++20, leveraging the Eigen library [15] for high-performance linear algebra with SIMD vectorization, and adopts a custom arena-based allocation scheme to minimize dynamic memory allocation during simulation steps, mitigating allocation overhead and memory fragmentation while ensuring predictable and stable performance.

B. Model Building

The user can configure custom simulation scenarios consisting of various single or articulated bodies. Currently, this is achieved through a C++ API that allows for the programmatic building of a hierarchical model structure. This tree-like high-level description reflects the kinematic connectivity of the robot and its environment, where each node contains the following essential information:

- **Mechanical properties** of each body and joint are defined to specify inertial characteristics, including mass and inertia, as well as joint configurations such as joint types, axes of motion, mechanical limits and frictions.
- **Geometric properties** define the visual or collision shapes associated with each body through diverse geometric representations, each assigned with surface attributes such as friction coefficient and compliance.
- **Visual properties** are assigned to enhance the quality of the visual output, managing attributes for the built-in renderer such as material colors, textures, and lighting.
- **Simulation options** consist of the parameters to tune the solver's performance and fidelity, including time step size, solver type, and convergence criteria.

To illustrate the ease of model definition, the following code snippet demonstrates the construction of a simulation environment comprising a robotic arm defined via a URDF file and a single box-shaped object.

```
auto builder = crisp::make_model_builder();
builder->addRobot({.path = "franka.urdf"});
auto& body = builder->addBody(
    {.pos = {0, 1, 1}, .mass = 10,
     .inertia = {1, 1, 1, 0, 0, 0}});
auto& geom = body.addGeom(
    {.type = crisp::geometry_e::box,
     .size = crisp::make_box_size(1, 1, 1)});
auto model = builder->build();
```

While certain parameters are mandatory to construct the valid simulation model, others are optional and provided with sensible default values.

Once the high-level model description is complete, it is compiled into a compact low-level data structure `model_t`, which organizes kinematic hierarchies into a flattened memory layout to maximize cache efficiency and optimize simulation performance. This static blueprint, which remains constant throughout the simulation, is then used to instantiate `data_t` as a mutable workspace that stores all time-varying quantities including generalized positions, velocities, contact informations, and solver states. This decoupled architecture, similar to that of MuJoCo, facilitates the multi-threaded execution of multiple simulations in parallel while sharing a single `model_t` instance, which aids in sampling-based control and high-fidelity dataset generation for robotic intelligence.

IV. COLLISION DETECTION

The collision detection module identifies contacts between geometries and provides the contact information required for the constraint solver. This section describes the supported geometric representations and the key collision detection algorithms implemented in CRISP.

A. Geometry Representations

CRISP supports a diverse range of geometric representations, allowing users to select the most appropriate one based on their specific requirements for computational efficiency and geometric fidelity. These include primitives (planes, spheres, capsules, boxes, and cylinders), convex hulls, and meshes, as well as function-based representations such as SDFs, DSFs, and our novel TDSFs. Each function-based representation is discussed in detail below.

1) *SDF*: An SDF defines the geometry through a scalar function which returns the signed distance from a point to the closest point on the surface of the shape. While SDFs offer the advantage of representing arbitrary non-convex shapes with a single geometric representation, they inherently increase the complexity of the collision detection problem. CRISP provides a flexible interface for defining SDF geometries, allowing users to register custom SDFs through callable functions that can be further parameterized for shape variations. To register a custom SDF, users need only provide a callable function that evaluates the signed-distance value at a given local query point. An analytic gradient may optionally be supplied; otherwise, CRISP approximates it using finite differences.

2) *DSF*: A support function $h : \mathbb{R}^3 \rightarrow \mathbb{R}$ for a convex set $\mathcal{C} \subset \mathbb{R}^3$ is defined by

$$h(x) = \sup_{s \in \mathcal{C}} x \cdot s$$

where $\cdot$ denotes the standard inner product. For closed convex sets, there exists a one-to-one correspondence between the convex set and its support function [39], allowing $\mathcal{C}$ to be uniquely defined by h . Building upon this, Lee et al. [21] introduced the DSF, which employs twice-differentiable support functions to represent strictly convex smooth shapes, which

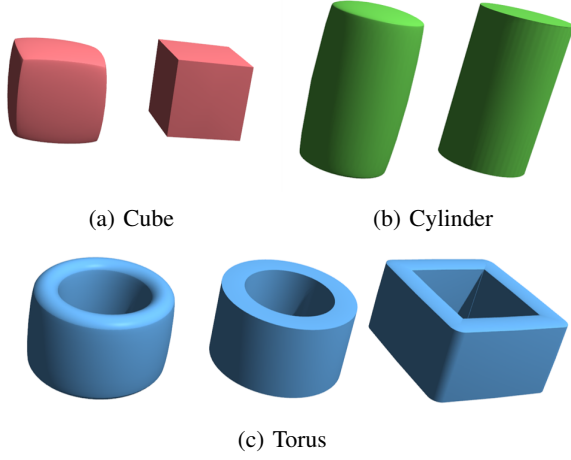


Fig. 3: **Visualizations of DSF and TDSF geometries.** It is possible to specify the degree of approximation by choosing the smoothness parameter, as described in Appendix A.

enables collision detection to be formulated as a differentiable optimization problem.

DSF offers several advantages over simple primitive shapes and convex hulls, as it formulates the collision detection problem as strongly convex, guaranteeing a unique contact point and normal which is further differentiable with respect to the geometry pose. From a simulation perspective, this ensures temporally consistent contact generation, making DSF particularly suitable for delicate manipulation between smooth convex shapes. Similar to SDF, DSF refers to a general class of shape functions, with CRISP currently supporting two specific DSF implementations: a vertex-based DSF to approximate a convex hull and an axis-based DSF to approximate a solid of revolution, as illustrated in Fig. 3. See Appendix A for detailed description of these classes.

3) *TDSF*: To further expand the range of shapes representable by DSFs, we introduce the toroidal DSF (TDSF), which defines a class of toroidal shapes by revolving a planar DSF along another planar DSF path, as illustrated in Fig. 3. Specifically, given two planar DSFs $h_1, h_2 : \mathbb{R}^2 \rightarrow \mathbb{R}$, a TDSF $h_T : \mathbb{R}^2 \times \mathbb{R}^2 \rightarrow \mathbb{R}$ is defined by

$$h_T(u, v) = v_1 h_1(u) + h_2(v)$$

where $v = (v_1, v_2) \in \mathbb{R}^2$. Although the representable shapes by TDSFs are limited to those with convex cross-sections, they can effectively model a variety of common geometries with a hole such as tori, and tubes while maintaining the computational efficiency and robustness advantages of DSF-based collision detection.

B. Collision Detection Algorithms

Following the standard practice in robotics simulation, CRISP first performs broad-phase filtering to prune non-colliding pairs, including axis-aligned bounding box checks, after which the remaining candidates are passed to the narrow-phase for precise collision detection. During the narrow-phase,

specialized algorithms are dispatched for each geometry pair according to the collision function table. As the efficiency and accuracy of collision detection algorithms vary, users may select the most appropriate geometry combination based on their specific requirements. Since some of the algorithms are standard or straightforward, we focus on describing algorithms associated with function-based representations and provide the complete collision function table in Appendix B.

1) *Mesh-SDF*: Following [29], CRISP identifies candidate mesh triangles that overlap with the SDF geometry and applies the Frank-Wolfe algorithm to selected candidates. This procedure is naturally parallelizable and identifies multiple contact points between two possibly non-convex shapes, ensuring robust collision detection even in complex configurations. For high-resolution meshes, this method may generate a large number of candidate contacts, which can be numerically challenging for the constraint solver; thus CRISP retains a bounded subset, favoring spatial distribution when applicable.

2) *SDF-SDF*: For SDF pairs, we adopt the same objective used in MuJoCo [38]: solve $\min_x \phi_1 + \phi_2 + |\max(\phi_1, \phi_2)|$ via multi-start gradient descent over the overlapping region. While this algorithm is applicable to arbitrary SDF pairs, it may fail to identify all contact points when the number of sampled initial points is insufficient.

3) *DSF-DSF*: For DSF pairs, we adopt the dedicated algorithm proposed in [1]. This algorithm utilizes Riemannian optimization to ensure superlinear convergence and theoretically guaranteed robustness, which is validated to exhibit superior speed and accuracy compared to standard convex-convex collision detection algorithm.

4) *SDF-DSF*: For SDF-DSF pairs, we exploit the fact that the surface unit normal of an SDF can be readily obtained from its gradient, allowing the SDF surface to be treated analogously to a DSF within the collision optimization. Based on this observation, we extend the DSF-DSF collision detection algorithm to SDF-DSF pairs, similarly to the extension used for DSF-TDSF collision detection.

5) *(T)DSF-TDSF*: For pairs involving TDSFs, we extend the DSF-DSF collision detection algorithm [1] by leveraging the special structure of TDSF. The extended algorithms also employ Riemannian optimization, ensuring accurate and efficient collision detection for both DSF-TDSF and TDSF-TDSF pairs. The DSF-TDSF algorithm is particularly advantageous over SDF-based methods for simulating peg-in-hole scenarios, as it provides consistent contact points and normals essential for physically accurate simulation.

V. CONSTRAINT SOLVING

Based on the contact information obtained from collision detection, the solver addresses the constrained dynamics of the system to compute the next state. CRISP currently provides two augmented-Lagrangian-based constraint solvers, CANAL and SubADMM. This section provides a concise description of these solvers; further algorithmic details are available in Appendix C and [22].

A. Problem Formulation

We consider following discretized constrained dynamics:

$$Av = b + J^T \lambda \quad \text{s.t. } (Jv, \lambda) \in \mathcal{S}_c \quad (1)$$

where $A \in \mathbb{R}^{n_v \times n_v}$ and $b \in \mathbb{R}^{n_v}$ represents the multi-body system dynamics [12], $v \in \mathbb{R}^{n_v}$ is the generalized representative velocity, $J \in \mathbb{R}^{n_c \times n_v}$ is the constraint Jacobian, and $\lambda \in \mathbb{R}^{n_c}$ is the constraint impulse. The set $\mathcal{S}_c$ represents the collective constraints of the system, which are categorized as joint limits, joint frictions, contact constraints (either rigid or compliant). All of these constraints can be formulated as complementarity conditions, as detailed in Appendix C, and are handled directly without numerical relaxations.

The AL-based solvers consider a decoupled reformulation of (1) by introducing an auxiliary variable $z \in \mathbb{R}^{n_c}$ as follows:

$$Av = b + J^T \lambda \quad \text{s.t. } (z, \lambda) \in \mathcal{S}_c, \quad Jv = z$$

with the corresponding AL defined as:

$$\mathcal{L} = \frac{1}{2} v^T A v - b^T v + g(z) + u^T (Jv - z) + \frac{\beta}{2} \|Jv - z\|^2 \quad (2)$$

where $u \in \mathbb{R}^{n_c}$ is the Lagrange multiplier, and $\beta > 0$ is the penalty weight, and g serves to enforce $(z, \lambda) \in \mathcal{S}_c$. At each iteration, the AL-based solver addresses the relaxed surrogate problem for the optimality of (2), and subsequently updates the multiplier u , gradually converging to the solution of the original problem (1). Since this AL approach maintains the feasibility of the surrogate problem, it enables stable and robust handling of complex complementarity constraints, particularly challenging rigid contacts, with least constraint violation even in poorly conditioned problems [10].

B. CANAL Solver

The aforementioned surrogate problem in AL can be expressed as the following semismooth equation:

$$r(v) = Av - b - J^T \Pi(-\beta Jv - u - \beta e) \quad (3)$$

where Π denotes the projection onto $\mathcal{S}_c$. CANAL addresses (3) via a cascaded Newton method, exploiting the integrability of the residual (3) into a strongly convex function by treating the De Saxcé corrections as constants borrowed from the previous AL iteration. This cascaded Newton structure enables CANAL to converge to a highly accurate solution of multi-contact NCP. We also note that a single AL iteration in CANAL corresponds to solving a convex compliant-contact subproblem with a Newton-type method, closely related to the convex contact formulations used in MuJoCo and Drake; thus CANAL can be viewed as an AL extension that iteratively refines such relaxations [22].

C. SubADMM Solver

While CANAL effectively resolves the multi-contact NCP, its reliance on Newton step necessitates the factorization of the Hessian matrix, which becomes computationally expensive for high degree-of-freedom (DoF) multi-body systems. As a scalable alternative, the SubADMM solver adopts an ADMM

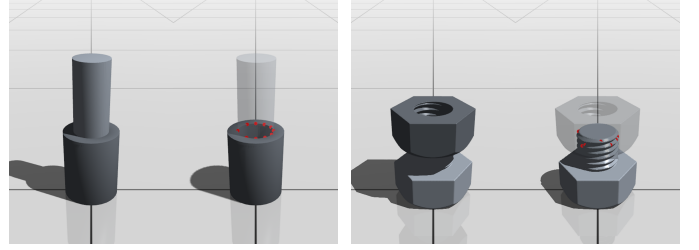


Fig. 4: **Visualization of assembly scenarios.** Initial configurations and examples of detected contact points. Left: peg insertion scenario. Right: bolt-nut assembly scenario.

approach [4] based on the decomposition of the dynamics (1) into subsystems (i.e., ground-rooted subtrees), each reformulated as follows:

$$A_j v_j = b_j + \sum_{i \in \mathcal{I}_j} J_{i,j}^T \lambda_i, \quad \text{s.t. } \left(\sum_{j \in \mathcal{J}_i} J_{i,j} v_j, \lambda_i \right) \in \mathcal{S}_{c,i}$$

where $\mathcal{I}_j \subset \{1, \dots, n_c\}$ and $\mathcal{J}_i \subset \{1, \dots, n_v\}$ are the index sets associated with the j -th subsystem and the i -th constraint, respectively, with $|\mathcal{J}_i|$ typically being 1 or 2.

Then the ADMM iteration process consists of performing an alternating update for each v_j and $z_{i,j}$ from the surrogate problem and updating the multiplier $u_{i,j}$, with each step naturally parallelizable as it is decoupled across subsystems or constraints. This structure allows SubADMM to quickly solve the high-DoF multi-contact NCP to a reasonable degree, yet CANAL yields superior accuracy via its second-order nature. Note that while Carpentier et al. [5] also proposed an ADMM-based NCP solver, their approach relies on a dual formulation to introduce slack variables for impulses, which precludes the subsystem-based variable splitting utilized in SubADMM.

VI. EVALUATION AND DEMONSTRATION

In this section, we present a comparative evaluation of CRISP against state-of-the-art simulators, MuJoCo and Isaac Sim, followed by demonstrations that highlight its simulation capabilities in complex robotic manipulation tasks. To quantitatively assess the physical accuracy of CRISP, we simulate several representative multi-contact scenarios and compare the results across simulators. For each comparative evaluation, we match geometric configurations, physical parameters, and time step sizes as closely as possible across simulators; additional setup details are provided in Appendix D. The experiments were conducted using MuJoCo v3.4.0 and Isaac Sim v4.5.0 on an AMD Ryzen 7 9800X3D CPU at 4.70 GHz.

A. Peg Insertion

The first evaluation scenario is a peg insertion task, where a cylindrical peg (radius 2.5 cm, height 10 cm) is inserted into a hole with a tight tolerance of 50–200 μm , as illustrated in Fig. 4. Although MuJoCo supports mesh–SDF collision, we report its analytic SDF–SDF configuration to match the SDF–SDF baseline in CRISP and to keep the comparison tied to the

Peg Insertion (Sec. VI-A)						
Tol. (μm)	CRISP (CANAL/SubADMM)				MuJoCo	Isaac Sim
	DSF-TDSF		SDF-SDF		SDF-SDF	Mesh-SDF
200	43.649	80.565	147.61	143.89	756.31	47.701
100	25.646	124.46	107.95	90.769	1349.9	22.551
50	37.844	43.503	79.334	106.09	1328.6	-
Bolt-Nut Assembly (Sec. VI-B)						
Tol. (μm)	CRISP (CANAL/SubADMM)				MuJoCo	Isaac Sim
	Mesh-SDF		SDF-SDF		SDF-SDF	Mesh-SDF
2000	9.5955	991.59	61.338	284.18	65.517	64.549
1000	9.1545	898.42	46.783	254.96	68.306	60.018
500	-	951.34	47.825	247.78	67.228	-

TABLE II: Average penetration depth across assembly scenarios. Units are in μm , and omitted entries indicate failed assemblies.

same analytic geometry. For Isaac Sim, we import the geometries as high-resolution meshes (on the order of 10^3 triangles) and use SDF mesh colliders generated from these meshes. CRISP supports both custom SDFs and a torus-specialized TDSF representation; accordingly, we evaluate DSF-TDSF (peg as DSF, hole as TDSF) and SDF-SDF geometric configurations with both the CANAL and SubADMM solvers. Since the clearances are extremely tight, we set the friction coefficient to 0.01 and apply a small angular perturbation of 0.5° without inducing jamming during insertion.

Since none of the compared simulators, including ours, are strictly intersection-free [24], we measure the average penetration depth throughout the insertion process as a quantitative metric of physical accuracy, as summarized in Table II. Overall, CRISP exhibits the lowest penetration across almost all tolerance levels. The MuJoCo SDF-SDF baseline shows the largest penetration, which can be attributed to two main factors: 1) its SDF-SDF collision detection relies on non-convex optimization, where insufficient initialization can miss contact points, particularly when the analytic SDFs exhibit discontinuous gradients as in the present setup; and 2) the Newton solver in MuJoCo employs numerical compliance to regularize contact constraints [44], inherently allowing penetration. These results suggest that CRISP benefits from the DSF-TDSF representation, which provides more stable contact detection than SDF-SDF in this scenario, as well as from direct NCP handling even in the SDF-SDF configuration. Isaac Sim, on the other hand, resolves penetration via positional constraint projection [33], yielding competitive metrics in this evaluation. However, since its SDFs are internally generated from voxelized representations, contact accuracy is limited, and we observe failure cases under the tightest tolerance (i.e., $50 \mu\text{m}$).

B. Bolt-Nut Assembly

We next consider a bolt-nut assembly task, which involves intensive multi-contact interactions along helical threads, as shown in Fig. 4. We primarily define the geometries using SDFs, based on analytic SDF formulations from the MuJoCo codebase [38]. For Isaac sim, we construct meshes with identical geometry, which are also evaluated in CRISP as Mesh-SDF case. We set the friction coefficient to zero to allow

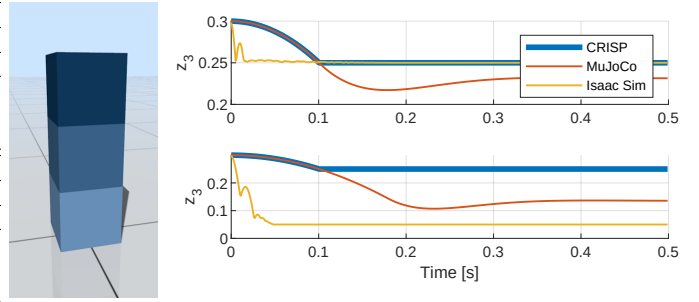


Fig. 5: Top-heavy stacking scenario. Left: Visualization of the scenario. Right: Height of the top block over time for mass ratios of 10 (top) and 100 (bottom).

gravity-driven assembly and measure the average penetration depth in the same manner, as summarized in Table II.

Consistent with the previous scenario, CRISP with the CANAL solver achieves the lowest penetration. In contrast, due to the intensive contact inherent in threaded assembly, we observe that the SubADMM solver suffers from limited convergence due to its first-order nature, leading to larger penetration. Unlike the peg insertion case, the SDF-SDF configuration in CRISP exhibits only a moderate advantage over MuJoCo in this scenario. We attribute this reduced advantage to the use of numerical differentiation for computing SDF gradients required by SDF-SDF collision detection, which leads to inaccuracies in contact point and normal estimation. When using the Mesh-SDF configuration, the mesh triangles provide effective guidance for initial contact points, enabling accurate collision detection and minimal penetration at moderate tolerances, while at the tightest tolerance the assembly fails due to limitations imposed by the mesh discretization.

C. Top-Heavy Stacking

In this scenario, we evaluate the robustness in handling ill-conditioned contact problems by stacking blocks with highly unbalanced mass ratios, where a heavier block is placed on top of lighter ones, all of which are cubic blocks with an edge length of 0.1 m, as illustrated in Fig. 5. We compare the height trajectories when the third block from the top is dropped slightly from above, while varying the mass ratio between adjacent blocks as 10 and 100. As shown in Fig. 5, CRISP successfully simulates the stacking process without instability for both mass ratios. In contrast, MuJoCo exhibits oscillatory behavior, particularly at a mass ratio of 100, which we attribute to regularization in its contact formulation that introduces artificial compliance [6]. Isaac Sim shows stable behavior in mass ratio of 10 with minor bouncing, but becomes unstable at a mass ratio of 100, likely due to the limited accuracy of its position-based contact solver [33] when dealing with ill-conditioned contacts.

D. Oblique Sliding

We further evaluate the accuracy of frictional contact handling through an oblique sliding test, where a block is placed on an inclined plane, as illustrated in Fig. 6. The inclination

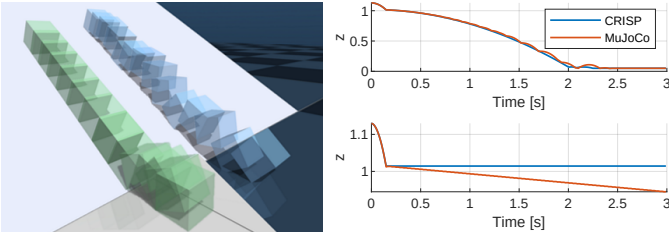


Fig. 6: **Oblique sliding scenario.** Left: Trajectories for the sliding case in CRISP (green) and MuJoCo (blue). Right: Height of the block over time for sliding case (top) and sticking case at the critical friction coefficient (bottom).

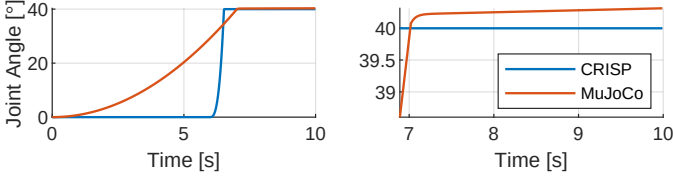


Fig. 7: **Joint constraint scenario.** Joint angle trajectories over time (left) and a magnified view near the final time step (right).

angle is set to 30° , with friction coefficients of $\tan(25^\circ)$ to induce sliding and $\tan(30^\circ)$ to achieve sticking. While CRISP yields the expected sliding and sticking behaviors, MuJoCo exhibits deviations in both cases: in the sliding case, the CCP formulation induces a *gliding* artifact [6] that causes the block to lift off the inclined plane, and even in the sticking case, its contact regularization in tangential direction leads to creeping motion despite the presence of friction. In contrast, Isaac Sim exhibits relatively stable behavior due to its position-level constraint projection; however, we observe that the simulation errors are inconsistent with respect to the initial conditions.

E. Joint Constraints

The last evaluation scenario focuses on the accurate enforcement of joint constraints. We simulate a simple door handle mechanism consisting of a revolute joint with joint limits of $\pm 40^\circ$ with friction, then apply a linearly increasing torque to the handle. As shown in Fig. 7, CRISP accurately enforces the joint limits and friction, with the handle coming to a stop just before reaching the limit. In contrast, MuJoCo fails to completely suppress the applied torque through joint friction, and its soft enforcement of joint limits leads to observable limit violations. We omit Isaac Sim from this comparison because an equivalent joint-friction setup was not available in our evaluation implementation.

F. Demonstration Scenarios

We now demonstrate the simulation capabilities of CRISP through robotic manipulation scenarios involving complex geometric configurations and rich multi-contact interactions.

1) *Gear Insertion and Driving:* We first consider a gear insertion and driving task, as illustrated in Fig. 1. In this scenario, a robotic arm inserts a gear into a housing, after which the gear is driven by an adjacent pre-assembled gear through

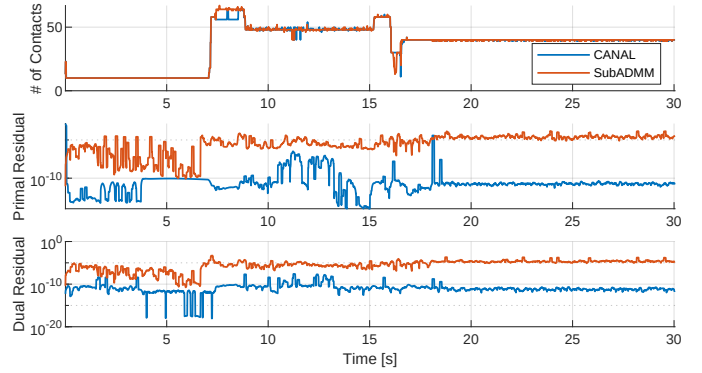


Fig. 8: **Results of gear insertion and driving simulation.** Number of contacts and solver residuals as a function of time.

sustained frictional contact. From a simulation perspective, this scenario is challenging due to the presence of simultaneous, contact-rich interactions among multiple object pairs during insertion and driving. The gear and housing are modeled using analytic SDFs, and the simulation capabilities are evaluated using both the CANAL and SubADMM solvers. As shown in Fig. 8, the evolution of contact constraints and solver residuals over time indicates stable and robust simulation behavior, with the primal residual $\|Jv - z\|$ measuring constraint violation and the dual residual $\|Av - b - J^T \lambda\|$ quantifying deviation from the system dynamics.

2) *Robotic Bolt-Nut Assembly:* As a second demonstration, we implement a robotic bolt-nut assembly task, shown in Fig. 1. In this setup, a robotic arm equipped with a continuous rotary gripper with the nut attached approaches a bolt fixed to the ground, and performs the assembly by aligning and rotating the gripper along the thread axis. Following the evaluation results in Sec. VI-B, we use analytic SDF and mesh to represent the bolt and nut, respectively, to accurately capture the contact interactions along the threads. This scenario is characterized by dense and stiff contact interactions, which impose tightly coupled contact constraints under limited effective degrees of freedom, yet are handled robustly within CRISP to achieve stable and successful simulation of the assembly process.

VII. CONCLUSION

In this paper, we present CRISP, a new simulation platform to achieve high physical fidelity in contact-rich robotic scenarios. Our simulator supports diverse geometric representations and accurate optimization-based collision detection, and combines multi-contact constraint modeling with robust AL-based constraint solvers. Extensive evaluations demonstrates that CRISP reliably handles challenging contact scenarios and achieves improved accuracy and stability compared to state-of-the-art simulators.

By consistently resolving tightly coupled multi-contact interactions without unphysical compromises, CRISP is particularly well suited for contact-intensive and tight-tolerance tasks such as bolting, insertion, and complex assembly, which are

common in industrial environments. These capabilities make CRISP a practical simulation platform for both control and learning-based robotic systems.

Since CRISP is designed as a flexible and extensible platform, we plan to further enhance its capabilities along several directions. Future work will extend supported collision pairs, improve solver performance through preconditioning and parallelization, incorporate additional physical phenomena such as soft-body dynamics, and further enhance scalability across larger contact-rich systems.

REFERENCES

- [1] Seoki An, Somang Lee, Jeongmin Lee, Sunkyung Park, and Dongjun Lee. Collision detection between smooth convex bodies via Riemannian optimization framework. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 12464–12471. IEEE, 2024.
- [2] Sheldon Andrews, Kenny Erleben, and Zachary Ferguson. Contact and friction simulation for computer graphics. In *ACM SIGGRAPH 2022 Courses*, pages 1–172. ACM, 2022.
- [3] Mihai Anitescu. Optimization-based simulation of non-smooth rigid multibody dynamics. *Mathematical Programming*, 105(1):113–143, 2006.
- [4] Stephan Boyd and Lieven Vandenbergh. *Convex Optimization*. Cambridge University Press, 2004.
- [5] Justin Carpentier, Quentin Le Lidec, and Louis Montaut. From compliant to rigid contact simulation: a unified and efficient approach. In *Robotics: Science and Systems*, 2024.
- [6] Alejandro M. Castro, Frank N. Permenter, and Xuchen Han. An unconstrained convex formulation of compliant contact. *IEEE Transactions on Robotics*, 39(2):1301–1320, 2023.
- [7] HeeSun Choi, Cindy Crump, Christian Duriez, Asher Elmquist, Gregory Hager, David Han, Frank Hearl, Jessica Hodgins, Abhinandan Jain, Frederick Leve, Chen Li, Franziska Meier, Dan Negrut, Ludovic Righetti, Alberto Rodriguez, Jie Tan, and Jeff Trinkle. On the use of simulation in robotics: Opportunities, challenges, and suggestions for moving forward. *Proceedings of the National Academy of Sciences*, 118(1):e1907856118, 2021.
- [8] NVIDIA Corporation. Colliders — Omni Physics, 2026. URL https://docs.omniverse.nvidia.com/kit/docs/omni_physics/latest/dev_guide/rigid_bodies_articulations/collision.html#createsdfcollider.
- [9] Erwin Coumans and Yunfei Bai. PyBullet, a Python module for physics simulation for games, robotics and machine learning, 2016–2021. URL <https://pybullet.org>.
- [10] Yu-Hong Dai and Liwei Zhang. The augmented Lagrangian method can approximately solve convex optimization with least constraint violation. *Mathematical Programming*, 200(2):633–667, 2023.
- [11] Evan Drumwright and Jeffrey C. Trinkle. *Humanoid Robotics: A Reference*, chapter Contact Simulation, pages 1877–1931. Springer, 2019.
- [12] Roy Featherstone. *Rigid Body Dynamics Algorithms*. Springer, 2008.
- [13] Sarah F. Frisken, Ronald N. Perry, Alyn P. Rockwood, and Thouis R. Jones. Adaptively sampled distance fields: A general representation of shape for computer graphics. In *27th Annual Conference on Computer graphics and interactive techniques*, pages 249–254. ACM, 2000.
- [14] Elmer G. Gilbert, Daniel W. Johnson, and S. Sathiya Keerthi. A fast procedure for computing the distance between complex objects in three-dimensional space. *IEEE Journal on Robotics and Automation*, 4(2):193–203, 2002.
- [15] Gaël Guennebaud, Benoît Jacob, et al. Eigen, 2010. URL <https://libeigen.gitlab.io>.
- [16] Mathew Halm. *Addressing stiffness-induced challenges in modeling and identification for rigid-body systems with friction and impacts*. PhD thesis, University of Pennsylvania, 2023.
- [17] Peter C. Horak and Jeff C. Trinkle. On the similarities and differences among contact models in robot simulation. *IEEE Robotics and Automation Letters*, 4(2):493–499, 2019.
- [18] Taylor A. Howell, Simon Le Cleac’h, Jan Brüdigam, J. Zico Kolter, Mac Schwager, and Zachary Manchester. Dojo: A differentiable physics engine for robotics. *arXiv preprint arXiv:2203.00806*, 2022.
- [19] Jemin Hwangbo, Joonho Lee, and Marco Hutter. Per-contact iteration method for solving contact dynamics. *IEEE Robotics and Automation Letters*, 3(2):895–902, 2018.
- [20] Danny M. Kaufman, Shinjiro Sueda, Doug L. James, and Dinesh K. Pai. Staggered projections for frictional contact in multibody systems. In *ACM SIGGRAPH Asia 2008 papers*, pages 1–11. ACM, 2008.
- [21] Jeongmin Lee, Minji Lee, and Dongjun Lee. Uncertain pose estimation during contact tasks using differentiable contact features. In *Robotics: Science and Systems*, 2023.
- [22] Jeongmin Lee, Minji Lee, Sunkyung Park, Jinhee Yun, and Dongjun Lee. Variations of augmented Lagrangian for robotic multi-contact simulation. *IEEE Transactions on Robotics*, 41:3852–3869, 2025.
- [23] Jeongseok Lee, Michael X. Grey, Sehoon Ha, Tobias Kunz, Sumit Jain, Yuting Ye, Siddhartha S. Srinivasa, Mike Stilman, and C. Karen Liu. DART: Dynamic animation and robotics toolkit. *Journal of Open Source Software*, 3(22):500, 2018.
- [24] Minchen Li, Zachary Ferguson, Teseo Schneider, Timothy R. Langlois, Denis Zorin, Daniele Panozzo, Chenfanfu Jiang, and Danny M. Kaufman. Incremental potential contact: intersection-and inversion-free, large-deformation dynamics. *ACM Transactions on Graphics*, 39(4):49, 2020.
- [25] Quentin Le Lidec, Wilson Jallet, Louis Montaut, Ivan Laptev, Cordelia Schmid, and Justin Carpentier. Contact models in robotics: a comparative analysis. *IEEE Transactions on Robotics*, 40:3716–3733, 2024.

- [26] John E. Lloyd. Fast implementation of Lemke’s algorithm for rigid body contact simulation. In *2005 IEEE International Conference on Robotics and Automation*, pages 4538–4543. IEEE, 2005.
- [27] Miles Macklin, Kenny Erleben, Matthias Müller, Nuttapong Chentanez, Stefan Jeschke, and Viktor Makoviychuk. Non-smooth newton methods for deformable multi-body dynamics. *ACM Transactions on Graphics*, 38(5):1–20, 2019.
- [28] Miles Macklin, Kier Storey, Michelle Lu, Pierre Terdiman, Nuttapong Chentanez, Stefan Jeschke, and Matthias Müller. Small steps in physics simulation. In *18th Annual ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, pages 1–7. ACM, 2019.
- [29] Miles Macklin, Kenny Erleben, Matthias Müller, Nuttapong Chentanez, Stefan Jeschke, and Zach Corse. Local optimization for robust signed distance field collision. *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, 3(1):1–17, 2020.
- [30] Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, and Gavriel State. Isaac gym: High performance gpu-based physics simulation for robot learning. *arXiv preprint arXiv:2108.10470*, 2021.
- [31] Khaled Mamou and Faouzi Ghorbel. A simple and efficient approach for 3D mesh approximate convex decomposition. In *2009 16th IEEE International Conference on Image Processing*, pages 3501–3504. IEEE, 2009.
- [32] Louis Montaut, Quentin Le Lidec, Vladimir Petrik, Josef Sivic, and Justin Carpentier. GJK++: Leveraging acceleration methods for faster collision detection. *IEEE Transactions on Robotics*, 40:2564–2581, 2024.
- [33] Matthias Müller, Miles Macklin, Nuttapong Chentanez, Stefan Jeschke, and Tae-Yong Kim. Detailed rigid body simulation with extended position based dynamics. *Computer Graphics Forum*, 39(8):101–112, 2020.
- [34] Yashraj Narang, Kier Storey, Ireteayo Akinola, Miles Macklin, Philipp Reist, Lukasz Wawrzyniak, Yunrong Guo, Adam Moravanszky, Gavriel State, Michelle Lu, Ankur Handa, and Dieter Fox. Factory: Fast contact for robotic assembly. In *Robotics: Science and Systems*, 2022.
- [35] NVIDIA et al. Isaac lab: A GPU-accelerated simulation framework for multi-modal robot learning. *arXiv preprint arXiv:2511.04831*, 2025.
- [36] NVIDIA et al. Cosmos world foundation model platform for Physical AI. *arXiv preprint arXiv:2501.03575*, 2025.
- [37] Jia Pan, Sachin Chitta, and Dinesh Manocha. FCL: A general purpose library for collision and proximity queries. In *2012 IEEE International Conference on Robotics and Automation*, pages 3859–3866. IEEE, 2012.
- [38] Alessio Quaglino. Extensions - MuJoCo Documentation, 2025. URL <https://mujoco.readthedocs.io/en/3.4.0/programming/extension.html#sdf>.
- [39] Ralph Tyrell Rockafellar. *Convex Analysis*. Princeton University Press, 1970.
- [40] Russ Smith. Open Dynamics Engine, 2008. URL <https://www.ode.org>.
- [41] Gary Snethen. Complex collision made simple. In *Game Programming Gems 7*, pages 165–178. Charles River Media, 2008.
- [42] David E. Stewart and Jeffrey C. Trinkle. An implicit time-stepping scheme for rigid body dynamics with inelastic collisions and coulomb friction. *International Journal for Numerical Methods in Engineering*, 39(15):2673–2691, 1996.
- [43] Russ Tedrake and the Drake Development Team. Drake: Model-based design and verification for robotics, 2019. URL <https://drake.mit.edu>.
- [44] Emanuel Todorov. Convex and analytically-invertible dynamics with contacts and constraints: Theory and implementation in MuJoCo. In *2014 IEEE International Conference on Robotics and Automation*, pages 6054–6061. IEEE, 2014.
- [45] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012.
- [46] Bolun Wang, Zachary Ferguson, Teseo Schneider, Xin Jiang, Marco Attene, and Daniele Panozzo. A large-scale benchmark and an inclusion-based algorithm for continuous collision detection. *ACM Transactions on Graphics*, 40(5):1–16, 2021.
- [47] Xinyue Wei, Minghua Liu, Zhan Ling, and Hao Su. Approximate convex decomposition for 3d meshes with collision-aware concavity and tree search. *ACM Transactions on Graphics*, 41(4):1–18, 2022.
- [48] Jaemin Yoon, Minji Lee, Dongwon Son, and Dongjun Lee. Fast and accurate data-driven simulation framework for contact-intensive tight-tolerance robotic assembly tasks. *arXiv preprint arXiv:2202.13098*, 2022.
- [49] Kevin Zakka, Baruch Tabanpour, Qiayuan Liao, Mustafa Haiderbhai, Samuel Holt, Jing Yuan Luo, Arthur Allshire, Erik Frey, Koushil Sreenath, Lueder A Kahrs, Carmelo Sferrazza, Yuval Tassa, and Pieter Abbeel. Demonstrating MuJoCo playground. In *Robotics: Science and Systems*, 2025.

	Plane	Sphere	Capsule	Box	Cylinder	Convex	Mesh	SDF	DSF	TDSF
Plane	–	Analytic				SF	BF	GD	SF	
Sphere			Analytic					Direct	RTR	
Capsule								Search		
Box					MPR		FW	GD	MPR	
Cylinder										
Convex							MPR	FW		
Mesh								GD		
SDF									RTR	
DSF										RTR
TDSF										RTR

TABLE III: **Collision function table in CRISP.** Each cell indicates the collision detection algorithm used for the corresponding geometry pair (–: cannot be defined, ×: not supported). Cell colors indicate a qualitative assessment of algorithmic performance (green: highly reliable, blue: recommended, yellow: neutral, red: use with caution).

APPENDIX

A. Classes of DSF

CRISP currently supports two classes of DSFs, as illustrated in Fig. 3. The first is the vertex-based DSF originally proposed in [21], which approximates a convex hull with vertices $\{v_1, \dots, v_n\} \subset \mathbb{R}^3$ and defined by

$$h(x) = \left(\sum_{i=1}^n \{\max(v_i \cdot x, 0)\}^p \right)^{\frac{1}{p}}$$

where $p > 2$ denotes the sharpness of the smoothed shape.

The second is the axis-based DSF proposed in [1], which corresponds to a smooth convex shape generated by rotating a convex hull defined on xz -plane with vertices $\{v_1, \dots, v_n\} \subset \mathbb{R}^2$ symmetric to z -axis and defined by

$$h(x) = \left(\sum_{i=1}^n \{\max(v_i \cdot x_{rz}, 0)\}^p \right)^{\frac{1}{p}}$$

where $x_{rz} := (\sqrt{x_1^2 + x_2^2 + \epsilon x_3^2}, x_3)$ and $\epsilon > 0$ determines the flatness at the poles.

B. Collision Function Table

For each geometry pair, CRISP dispatches the most suitable collision detection algorithm based on the characteristics of the involved geometries, as summarized in Table III.

1) *Convex Collisions:* Collision detection between convex geometries can be formulated as a convex optimization problem, enabling efficient algorithms with strong convergence properties [14]. While a single contact generally suffices for strictly convex pairs, weakly convex shapes such as boxes and convex hulls may require multiple contacts to represent edge or face contact robustly [11]. CRISP uses specialized analytic routines for primitive pairs, including the separating axis theorem for box–box collisions [4], and support-function (SF) queries when a plane is paired with a support-mapped geometry. For general convex pairs without a specialized routine, CRISP employs Minkowski portal refinement (MPR) [41] with rotational perturbations to generate multiple contacts when needed, while DSF–DSF pairs use the dedicated Riemannian trust-region (RTR) method [1].

2) *Non-Convex Collisions:* Collision detection involving non-convex geometries is more challenging because multiple local solutions may exist, motivating pair-specific algorithms that exploit the available geometric structure. These include brute-force sampling (BF) for simple mesh pairs, Frank–Wolfe (FW) for Mesh–SDF, gradient descent (GD) for SDF–SDF, and RTR optimization for SDF–DSF and selected TDSF pairs; see Sec. IV and Table III. For geometry pairs without a dedicated non-convex routine, a convex-hull fallback may be used where applicable, while the remaining combinations are not currently supported.

C. Algorithmic Details on Constraint Solving

The constraints supported in CRISP are categorized as follows:

- **Limit:** For joints defined with upper and lower bounds, the following hard inequality constraints are imposed:

$$0 \leq \lambda_i \perp J_i v + e_i \geq 0$$

where $e_i \in \mathbb{R}$ and $J_i \in \mathbb{R}^{1 \times n_v}$ are the error and Jacobian for joint limit constraint, and $\perp$ denotes complementarity.

- **Friction:** For joints with friction parameters, the following dry friction constraint is imposed:

$$\begin{cases} |\lambda_i| \leq b_i & \text{if } J_i v = 0 \\ \lambda_i = -b_i J_i v / |J_i v| & \text{if } J_i v \neq 0 \end{cases}$$

where $J_i \in \mathbb{R}^{1 \times n_v}$ is the Jacobian for joint friction constraint and $b_i > 0$ is the maximum friction impulse.

- **Contact:** The contact condition is the most challenging constraint as it is characterized by a complex nonlinear complementarity relation. We employ the following extended Signorini-Coulomb condition with compliance:

$$\mathcal{C}_i \ni \lambda_i \perp J_i v + e_i + p_i + R_i \lambda_i \in \mathcal{C}_i^* \quad (4)$$

where $J_i = [J_{i,t}; J_{i,n}] \in \mathbb{R}^{3 \times n_v}$ and $e_i = [0; 0; e_{i,n}] \in \mathbb{R}^3$ are the contact Jacobian and error consisting of tangential and normal components, $p_i = [0; 0; \mu_i \|J_{i,t} v\|]$ is the De Saxcé correction term, $\mathcal{C}_i$ and $\mathcal{C}_i^*$ are the friction cone and its dual with coefficient $\mu_i > 0$, and

$R_i = \text{diag}(0, 0, c_i)$ introduces compliance $c_i \geq 0$, with $c_i = 0$ reducing to the rigid case.

For the AL defined by (2), the AL iteration step consists of solving the surrogate problem for optimality conditions (5), (6) and updating the multiplier (7) as follows:

$$(A + \beta J^T J)v = b + J^T(\beta z - u) \quad (5)$$

$$\beta z = \beta Jv + u + \lambda, \quad \text{s.t. } (z, \lambda) \in \mathcal{S}_c \quad (6)$$

$$u \leftarrow u + \beta(Jv - z) \quad (7)$$

We note that the closed-form solution to (6) can be derived via the following projection operations for each constraint type:

$$\lambda_i = \Pi_i(-\beta J_i v - u_i - \beta e_i) \quad (8)$$

For instance, regarding the contact constraint in (4), the projection Π_i corresponds to the projection onto the friction cone $\mathcal{C}_i$ while preserving the non-negative normal component:

$$\lambda_{i,n} = \begin{cases} \Pi_{\geq 0}(-\beta J_{i,n} v - u_{i,n} - \beta e_{i,n}) & \text{if } c_i = 0 \\ \Pi_{\geq 0}(-c_i^{-1}(J_{i,n} v + \beta^{-1} u_{i,n} + e_{i,n})) & \text{if } c_i > 0 \end{cases}$$

$$\lambda_{i,t} = \Pi_{\|\cdot\| \leq \mu_i \lambda_{i,n}}(-J_{i,t} v - \beta^{-1} u_{i,t})$$

where $\Pi_{\geq 0}$ and $\Pi_{\|\cdot\| \leq r}$ denote the projection onto the non-negative reals and the ball of radius $r > 0$, respectively. By substituting (8) into (5), we obtain the semismooth equation (3) for the surrogate problem which is addressed in CANAL solver.

D. Evaluation Setups

This section describes the experimental configurations for each scenario. To ensure a fair comparison, we configure the geometric and physical settings to be as identical as possible across all simulators. Unless otherwise specified, the simulation parameters were kept at their default values.

1) *Peg Insertion*: In this scenario, we evaluate the simulators by dropping the peg from a position directly above the hole with a slight tilt of 0.5° . To ensure geometric equivalence, both MuJoCo and CRISP utilize analytic SDFs for the peg and the hole, while Isaac Sim uses high-resolution meshes with the voxel SDF resolution set to its maximum to minimize discretization errors. When CRISP employs DSF and TDSF geometries, the sharpness parameter is set to $p = 500$ to ensure that any geometric deviations from surface smoothing remain negligible, thereby preserving the intended sharp features. We use a time step of $\Delta t = 2$ ms across all engines. To quantify the penetration depth, we consider a reduced 2D planar representation and calculate the maximum penetration depth for each time step during the insertion process and average these values over time.

2) *Bolt-Nut Assembly*: For this scenario, we utilize analytic SDFs provided by the MuJoCo codebase [38], where the tolerance is controlled by adjusting the radius parameter of the bolt. Specifically, for evaluations in Isaac Sim and CRISP (with Mesh-SDF collisions), we generate high-resolution meshes that precisely match the thread geometry defined by the analytic SDFs. In Isaac Sim, we further set the voxel SDF

resolution to its maximum to minimize discretization errors. Since the complex geometries at the thread entrance and the connection to the bolt head cannot be perfectly matched between meshes and analytic SDFs, we collect data only during the middle segment of the engagement, where the nut is fully engaged with the threads. A time step of $\Delta t = 10$ ms is used for this scenario. Unlike the previous case where an explicit measurement of penetration depth is possible, the complex helical geometry of the threads precludes such a direct calculation; we therefore record the maximum penetration among all contact points at each time step and calculate their average for comparison.

3) *Top-Heavy Stacking*: In this scenario, we evaluate the stacking stability by varying the mass ratio of the top and bottom blocks to 10 and 100, while keeping the middle block at 1 kg. To ensure the stack remains stable without slipping, the friction coefficient is set to 5.0, and a time step of $\Delta t = 1$ ms is used. We note that while MuJoCo exhibits noticeable penetration in this setup, such effects can be reduced by tuning `solimp`, but finite penetration may remain under MuJoCo's soft-contact formulation.

We observe that since this configuration is underdetermined in terms of contact forces, CANAL and SubADMM exhibit different force distribution characteristics: while SubADMM tends to distribute contact forces across the contact surface, CANAL shows a tendency to concentrate forces at specific contact points. Despite these different behaviors in force distribution, both methods successfully maintain a stable stack under all tested mass ratios.

4) *Oblique Sliding*: In the oblique sliding scenario, we evaluate the accuracy for frictional behavior by dropping a 1 kg block, rotated to align with the slope's inclination, from a height of 0.1 m above the plane. We use a time step of $\Delta t = 10$ ms for this evaluation. Similar to the previous scenarios, we note that gliding or creeping in MuJoCo can be mitigated by tuning parameters such as `impratio` or `solimp`, but may persist under the compliant convex contact formulation used by MuJoCo.

5) *Joint Constraints*: In this scenario, we model a door handle mechanism to evaluate the precision of joint constraints. The joint is configured with a friction torque of 0.06 N m and limits set at $\pm 40^\circ$. To isolate the joint's response to external inputs, we disable gravity and attach a force-controlled actuator, with the input torque increasing linearly over time. A time step of $\Delta t = 10$ ms is applied across both simulators.

6) *Demonstration Scenarios*: For both demonstration scenarios, we utilize a time step of $\Delta t = 10$ ms. To control the robotic manipulator, we employ a low-level PD controller for each joint where the desired joint angles are determined by interpolating between a set of predefined keyframes.